In [17]:

```
# accuracy= TP +TN/ (TP+TN+FP+FN) 40+30/100
# presision=  TP/(TP+FP)   #when FP is costly   Eg:- email spam
# recall= TP/(TP+FN)   # when FN is costly      Eg:- deseases prediction
# f1 score= harmonic mean of recall and precision
```

In [18]:

```
# spam === 100
#   0     1          0                            1    predicted
# 0[30    20  ]   [True positive            false positive]
# 1[10    40  ]    [false negitive           True Negitive]

# actual
```

In [ ]:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor, plot_tree
import matplotlib.pyplot as plt
```

In [20]:

```
df= pd.read_csv("carprices.csv")
```

In [21]:

```
df
```

Out[21]:

	Mileage	Age(yrs)	Sell Price($)
0	69000	6	18000
1	35000	3	34000
2	57000	5	26100
3	22500	2	40000
4	46000	4	31500
5	59000	5	26750
6	52000	5	32000
7	72000	6	19300
8	91000	8	12000
9	67000	6	22000
10	83000	7	18700
11	79000	7	19500
12	59000	5	26000
13	58780	4	27500
14	82450	7	19400
15	25400	3	35000
16	28000	2	35500
17	69000	5	19700
18	87600	8	12800
19	52000	5	28200

In [22]:
```
x= df[['Mileage', 'Age(yrs)']]
y= df['Sell Price($)']
```

In [23]:
```
x_train, x_test, y_train, y_test= train_test_split(x,y, test_size=0.2)
```

In [24]:
```
model = DecisionTreeRegressor(max_depth=5)
```

In [25]:
```
model.fit(x_train, y_train)
```

Out[25]:

▾ DecisionTreeRegressor

Parameters

criterion	'squared_error'
splitter	'best'
max_depth	5
min_samples_split	2
min_samples_leaf	1
min_weight_fraction_leaf	0.0
max_features	None
random_state	None
max_leaf_nodes	None
min_impurity_decrease	0.0
ccp_alpha	0.0
monotonic_cst	None

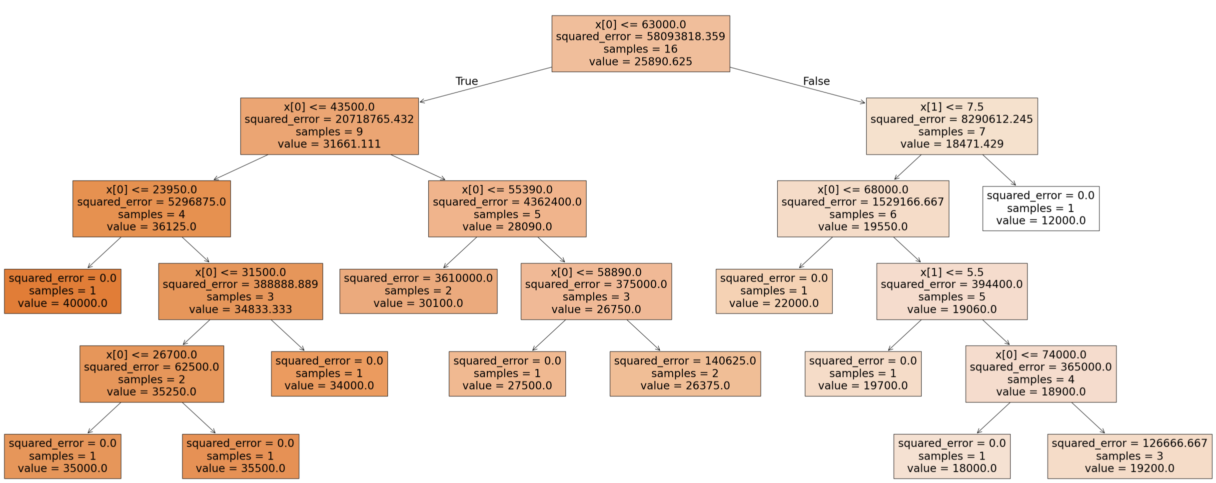
In [26]:
```python
model.score(x_test, y_test)
```

Out[26]: 0.968476931418412

In [27]:
```python
model.score(x_train, y_train)
```

Out[27]: 0.9915209889982983

In [31]:
```python
plt.figure(figsize=(50,20))
plot_tree(model, filled=True)
plt.show()
```



In [37]:
```
x_train.Mileage.mean()
```

Out[37]:
```
np.float64(58258.125)
```

In [ ]:
```
# x.Mileage.mean()
```

Out[ ]:
```
np.float64(59736.5)
```

In [38]:
```
x_train.Mileage
```

Out[38]:
```
15    25400
5     59000
9     67000
1     35000
13    58780
10    83000
16    28000
6     52000
3     22500
14    82450
0     69000
19    52000
8     91000
12    59000
11    79000
17    69000
Name: Mileage, dtype: int64
```

http://www.saedsayad.com/decision_tree_reg.htm

In [ ]:
```

```